

Tarea 1: Computación de Alto Rendimiento (IIC3533)

Josefina Abbott Cerda, José Ignacio Salas Coeymans y Borja Smith Vergara

11 de septiembre de 2026

(a) Generación de datos sintéticos

Para generar los datos sintéticos, seguimos los siguientes pasos:

1. Generar los datos, usando el generador aleatorio de NumPy (`np.random.default_rng`) fijando una semilla base, nosotros la establecimos como 1111.
2. Generar el vector de coeficientes reales $\beta^* \in \mathbb{R}^{301}$ muestreando de una distribución normal estándar $\mathcal{N}(0, 1)$.
3. Crear una matriz base de 10000×300 con valores independientes $\mathcal{N}(0, 1)$.
4. Concatenar horizontalmente un vector columna de unos a la matriz base para incorporar el intercepto (β_0), obteniendo así la matriz de diseño $X \in \mathbb{R}^{10000 \times 301}$.
5. Calcular el vector de respuesta $y \in \mathbb{R}^{10000}$ con el producto matricial $y = X\beta^* + \varepsilon$, donde ε es un vector de ruido con 10000 muestras i.i.d. de $\mathcal{N}(0, 1)$.

Para correr el script de generación de datos, se puede ejecutar el siguiente comando en la terminal:

```
python src/data_generation.py
```

(b) Implementaciones de bootstrapping

Evolución de `bs_auto.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Utiliza la configuración por defecto de scikit-learn. Emplea `LinearRegression()` (que calcula un intercepto internamente) y extrae los coeficientes iterando con un ciclo `for` estándar y `.append()`.
- **Versión 2:** Introduce `fit_intercept=False`. Dado que la matriz de diseño X ya incluye una columna de unos pre-concatenada[cite: 1], esto evita el cálculo de un intercepto redundante.

- **Versión 3:** Reemplaza el ciclo `for` por una comprensión de listas (*list comprehension*) vectorizada para la extracción de coeficientes, mejorando la asignación de memoria.
- **Versión 4:** Añade explícitamente el parámetro `bootstrap=True` al `BaggingRegressor` para garantizar la correctitud del algoritmo (muestreo con reemplazo)[cite: 1], manteniendo la comprensión de listas.
- **Versión 5:** Mantiene los parámetros óptimos del modelo, pero revierte la extracción de coeficientes al ciclo `for` con `.append()` para contrastar el manejo de memoria final.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	Intercepto activo, ciclo <code>for</code>	4.7474 s
v2	Sin intercepto, ciclo <code>for</code>	5.1836 s
v3	Sin intercepto, list comprehension	5.3599 s
v4	Sin inter., list comp., bootstrap explícito	4.2048 s
v5	Sin inter., ciclo <code>for</code> , bootstrap explícito	4.1570 s

Cuadro 1: Tiempos de ejecución empíricos para distintas iteraciones de `bs_auto.py`.

3. Análisis de la versión óptima

La **Versión 4** es computacional y estructuralmente la más óptima.

En primer lugar, al definir `fit_intercept=False`, se elimina una operación matricial redundante que `scikit-learn` realizaría en cada uno de los $B = 48$ remuestreos, ya que la matriz X construida ya contempla el intercepto en su primera columna[cite: 1]. Además, declarar `bootstrap=True` asegura el rigor matemático del remuestreo exigido[cite: 1], protegiendo el código contra posibles cambios en los valores por defecto de la librería.

Si bien empíricamente la Versión 5 registró un tiempo marginalmente inferior en esta ejecución específica (4.15s vs 4.20s), esta diferencia de ~ 0.05 segundos es atribuible a la varianza natural del *scheduling* de la CPU en el sistema operativo. A nivel de arquitectura de software, la Versión 4 es superior porque utiliza una comprensión de listas (`[est.coef_ for est in ...]`) en lugar de un `.append()` dinámico. Esto permite que el motor en C de Python asigne el bloque de memoria necesario de una sola vez, en lugar de redimensionar la lista continuamente en cada iteración, previniendo cuellos de botella en la memoria RAM a medida que el proceso escala.

Evolución de `bs_sklern.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Utiliza el generador aleatorio global (`np.random.choice`) dentro de cada proceso trabajador. Esto provoca que procesos iniciados simultáneamente compartan la misma semilla y extraigan las mismas muestras, arruinando la independencia del bootstrapping. Además, mantiene activado el cálculo del intercepto por defecto.

- **Versión 2:** Aísla la generación de números aleatorios asignando una semilla determinista única (`seed_b`) a cada proceso mediante `np.random.default_rng(seed_b)`. Esto incrementa levemente el tiempo de ejecución por el costo de instanciar 48 generadores distintos, pero es matemáticamente obligatorio para garantizar la validez estadística.
- **Versión 3:** Introduce `fit_intercept=False`. Al igual que en `bs_auto.py`, esto elimina el procesamiento redundante de la matriz, reduciendo los tiempos de ejecución notablemente. Adicionalmente, el retorno estricto de `model.coef_` en lugar del objeto modelo completo evita cuellos de botella por serialización (*pickling*) durante la comunicación entre procesos de `joblib`.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	RNG global (incorrecto), intercepto activo	5.9958 s
v2	RNG independiente (correcto), intercepto activo	7.7394 s
v3	RNG independiente, sin intercepto	5.5406 s

Cuadro 2: Tiempos de ejecución empíricos para distintas iteraciones de `bs_sklearn.py`.

3. Análisis de la versión óptima

La **Versión 3** representa la implementación correcta y más eficiente utilizando scikit-learn de forma explícita. El manejo independiente de las semillas elimina las condiciones de carrera paralela, mientras que la desactivación del intercepto evita cálculos innecesarios sobre la matriz X , devolviendo el tiempo de ejecución a niveles óptimos (~ 5.54 s) sin comprometer la correctitud del algoritmo iterativo.

Evolución de `bs_numpy.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Traduce literalmente la fórmula de Mínimos Cuadrados Ordinarios entregada en el enunciado: $\hat{\beta} = (X^T X)^{-1} X^T y$ [cite: 1]. Utiliza `np.linalg.inv()` para calcular explícitamente la matriz inversa antes de realizar las multiplicaciones matriciales.
- **Versión 2:** Reemplaza la inversión explícita reescribiendo el problema como la resolución de un sistema de ecuaciones lineales: $(X^T X)\hat{\beta} = X^T y$ [cite: 1]. Utiliza `np.linalg.solve()` para encontrar los coeficientes de manera directa.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	Cálculo explícito con <code>np.linalg.inv()</code>	1.0937 s
v2	Resolución de sistema con <code>np.linalg.solve()</code>	1.0204 s

Cuadro 3: Tiempos de ejecución empíricos para distintas iteraciones de `bs_numpy.py`.

3. Análisis de la versión óptima

La **Versión 2** es la implementación óptima y definitiva para esta sección.

Aunque la diferencia de tiempo empírica en una sola ejecución con $p = 1$ parece marginal (~ 0.07 segundos), el cambio posee un peso teórico y arquitectónico fundamental. Calcular explícitamente la inversa de una matriz mediante `np.linalg.inv()` es computacionalmente ineficiente y altamente susceptible a errores de redondeo de punto flotante a medida que la dimensionalidad aumenta.

Al utilizar `np.linalg.solve()`, NumPy delega la operación a subrutinas optimizadas de álgebra lineal (LAPACK), las cuales resuelven el sistema de ecuaciones $(X^T X)\hat{\beta} = X^T y$ [cite: 1] mediante métodos de factorización directa (como descomposición LU o Cholesky) sin llegar a instanciar la matriz inversa en la memoria. Esto garantiza una mayor estabilidad numérica y previene cuellos de botella algorítmicos, preparándonos correctamente para los experimentos de escalabilidad con p_{max} procesos [cite: 1].

Para correr las implementaciones, se pueden ejecutar los siguientes comandos en la terminal:

```
python src/bs_auto.py
python src/bs_sklearn.py
python src/bs_numpy.py
```

(c) Correctitud y reproducibilidad

Consistencia con β^* : Las tres versiones producen intervalos de confianza empíricamente consistentes con los coeficientes verdaderos β^* . La implementación automatizada (`bs_auto.py`) logra una cobertura del 93.69 %, mientras que las implementaciones manuales (`bs_sklearn.py` y `bs_numpy.py`) alcanzan un 91.69 %. Aunque la cobertura se encuentra ligeramente por debajo del 95 % teórico, esto es un comportamiento esperado debido al reducido número de remuestreos exigido ($B = 48$) [cite: 1]. Pese a ello, las tres iteraciones capturan exitosamente la inmensa mayoría de los coeficientes, confirmando la validez estructural del algoritmo.

Equivalencia entre versiones:

- `bs_sklearn.py` y `bs_numpy.py` son estrictamente equivalentes. Al ejecutar la validación, se comprobó que ambas implementaciones son matemáticamente idénticas. Dado que utilizan el mismo arreglo de semillas (`seed_b`) para el generador `np.random.default_rng`, extraen exactamente las mismas muestras en cada iteración b . En consecuencia, sus estimaciones convergen sin discrepancias.
- `bs_auto.py` es equivalente a nivel estadístico (logrando una cobertura similar del $\sim 93\%$), pero sus intervalos exactos difieren. Esto ocurre porque `BaggingRegressor` administra su propio estado aleatorio interno para el remuestreo de manera opaca, generando secuencias de índices distintas a las provistas en las versiones manuales.

Condiciones de reproducibilidad: Para que los resultados sean estrictamente reproducibles entre distintas ejecuciones, se deben cumplir tres condiciones fundamentales:

1. Fijar la semilla del generador inicial utilizado para la creación sintética de la matriz X y el vector y [cite: 1].
2. Instanciar generadores aleatorios locales y aislados (`default_rng`) dentro de cada proceso trabajador de `joblib` para anular cualquier condición de carrera en la memoria compartida.
3. Garantizar una asignación determinista de semillas. El mapeo entre la iteración bootstrap b y su semilla debe ser invariante (por ejemplo, mediante una lista secuencial estática), asegurando que el remuestreo sea independiente del número de procesos p utilizados o del orden de asignación del sistema operativo.

Para verificar la correctitud y reproducibilidad de las implementaciones, se puede ejecutar el siguiente comando en la terminal:

```
python src/correctness.py
```

(d) Backend multiprocessing de joblib

Creación de procesos y asignación de memoria: En entornos Linux, como el habilitado a través de WSL, el backend de `joblib` utiliza llamadas al sistema operativo (típicamente variantes de `fork` mediante su gestor interno `loky`) para generar procesos hijos a partir del script principal. Al crear estos nuevos procesos, el sistema operativo no realiza una copia inmediata y exhaustiva de la memoria RAM del proceso padre. En su lugar, emplea una técnica de gestión de memoria denominada *Copy-on-Write* (CoW). Bajo este esquema, los procesos hijos recién creados reciben direcciones virtuales que apuntan a las mismas páginas de memoria física del proceso original. El sistema operativo solo asigna memoria nueva y duplica la información si un proceso hijo intenta sobrescribir o modificar los datos.

Comportamiento de los arreglos X e y : Al lanzar p procesos, los arreglos sintéticos X e y no se replican p veces en la RAM del computador[cite: 1]. Dado que la lógica del bootstrapping implementada únicamente requiere leer secciones específicas de los arreglos mediante indexación (`X[indices]`), las estructuras originales permanecen intactas y tratadas como datos de solo lectura. Gracias al mecanismo *Copy-on-Write*, los p trabajadores paralelos acceden simultáneamente a la única copia física de X e y que reside en la memoria del proceso principal. Esto previene un desbordamiento masivo de la memoria RAM (impidiendo que $X \in \mathbb{R}^{10000 \times 301}$ se copie innecesariamente[cite: 1]) y minimiza el *overhead* temporal al instanciar las tareas en paralelo.

- (e) Actividad del sistema y oversubscription
- (f) Tiempos de ejecución en función de p
- (g) Speedup $S(p)$ y Eficiencia $E(p)$
- (h) Overhead $T_o(p)$
- (i) Variación de procesos y threads internos
- (j) Comparación entre computadores

Declaración de uso de Inteligencia Artificial

En el desarrollo de esta tarea, se utilizaron modelos de lenguaje de Inteligencia Artificial como asistente de programación, estructuración y estudio (Gemini, ..., ...). Específicamente, la herramienta fue empleada para:

- **Optimización algorítmica:** Guiar la evolución iterativa del código (de implementaciones ingenuas a óptimas), sugiriendo mejoras como la vectorización de ciclos, el manejo determinista de semillas en `joblib` y la sustitución de inversiones matriciales por resolución de sistemas lineales (`np.linalg.solve`).
- **Edición y formato:** Asistir en la estructuración de este documento en $\text{\LaTeX}$.

Todo el código, los tiempos de ejecución y las explicaciones teóricas generadas con el apoyo de la herramienta fueron revisados, ejecutados y validados empíricamente por los integrantes del grupo para asegurar su total correctitud y cumplimiento con las exigencias del enunciado.